

Изучение существующих методов аппроксимации

Для выбора оптимального подхода, проведём обзор основных классов методов, которые потенциально применимы к нашему кейсу:

1. Линейная и полиномиальная регрессия

- *Суть:* Модель пытается представить целевую метрику как сумму входных признаков с определёнными весами
- *Плюсы и минусы:* Быстро обучается, легко интерпретируется, но чистая линейная модель не может улавливать сложные нелинейные связи без ручного преобразования

2. Древовидные ансамбли (Random Forest, Gradient Boosting)

- *Суть:* Алгоритмы строят ансамбль решающих деревьев, где каждое дерево принимает решение на основе последовательных условий
- *Плюсы и минусы:* Отлично работают с нелинейными данными и пороговыми значениями, устойчивы к выбросам. Не умеют предсказывать значения за пределами границ обучающей выборки

3. Методы опорных векторов (SVR)

- *Суть:* Переносит данные в более высокое пространство размерностей, где пытается построить разделяющую гиперплоскость
- *Плюсы и минусы:* Хорошо работает на небольших выборках, но долго обучается при росте количества данных и требует сложной настройки параметров

4. Нейронные сети

- *Суть:* Набор связанных слоёв нейронов с нелинейными функциями активации
- *Плюсы и минусы:* Могут выучить любую зависимость, но требуют очень много данных для обучения

Выбор методов для экспериментов и их обоснование

На основе анализа специфики задачи для проведения экспериментов были выбраны два метода: линейная регрессия (с логарифмированием признаков, тк значения W , D различаются на порядки) (Baseline) и случайный лес (основной метод)

Обоснование выбора:

- **Ожидаемая форма зависимостей**

Из теории алгоритмов и структуры исследуемых подходов (A , B , C) можно выделить следующие особенности:

- **Нелинейность:** В подходе A время поиска зависит от глубины D и ширины W как $O(D \cdot \log W)$ или $O(D \cdot W)$. Это явно нелинейная зависимость. Чтобы линейная регрессия могла её уловить, мы применим логарифмирование признаков
- **Экспоненциальный рост:** В подходах B и C групповые операции вроде `ls` или `find` зависят от размера поддеревья, который при увеличении коэффициента заполнения F может расти экспоненциально

Из-за нелинейной природы и взаимного влияния параметров Random Forest выбран в качестве основного метода. За счёт своей структуры он может автоматически подстраиваться под особенности функций

- **Количество доступных данных**

Согласно плану формирования сетки, мы имеем **требуется уточнений:**

$$N = \text{точек измерений на один подход}$$

- **Требования к интерпретируемости**

- **Линейная регрессия** даёт максимальную интерпретируемость: знак и величина коэффициента перед признаком показывают, насколько сильно и в какую сторону меняется метрика при изменении признака
- **Random Forest** менее прозрачен, но он предоставляет оценку важности признаков. С помощью неё мы сможем построить график и наглядно показать, какой из параметров вносит наибольший вклад

- **Вычислительные ресурсы**

Поскольку обученные модели будут использоваться внутри рекомендателя, хотелось бы иметь хорошую скорость на этапе предсказания

- Линейная регрессия выполняет только операцию скалярного произведения, что работает мгновенно
- Обученный Random Forest тратит время только на проход по веткам деревьев фиксированной глубины, что также имеет константную сложность $O(1)$ относительно объема данных

Оба метода легковесны и не потребуют мощных вычислительных ресурсов ни при обучении, ни при работе рекомендателя.

- **Наличие корреляций между целевыми метриками**

Исходя из физики работы вычислительных систем, между нашими целевыми метриками ожидается сильная взаимосвязь:

- Пропускная способность (Throughput) и средняя латентность (Latency) связаны обратно пропорционально, чем выше задержки, тем меньше операций в секунду обрабатывает система
- 99-й процентиль латентности очевидно связан со средней латентностью, но характер этой связи **требует уточнения после получения реальных данных**
- Потребление памяти (Memory) теоретически не должно напрямую зависеть от временных характеристик. В основном будет зависеть от структуры данных (D, W, F)

Точный характер и сила корреляций требуют уточнения на этапе анализа собранных данных

Проектирование признаков

Чтобы повысить точность аппроксимации и помочь линейной модели лучше улавливать особенности файловых систем, был проведён синтез новых нелинейных признаков. Базовый набор параметров включает в себя: глубину дерева (D), ширину папки (W), общее количество файлов (F) и относительную частоту вызова конкретных операций op_i

Математическое обоснование новых признаков

1. Логарифмирование структурных параметров ($\log D, \log W$)

Входные параметры структуры данных могут различаться на несколько порядков. Из-за этого линейная регрессия начинает сильно ошибаться на краях выборки. Так же из курса “Алгоритмы и ст” известно, что время поиска элемента в структуре часто зависит от параметров нелинейно (например, $O(D \cdot \log W)$). Это позволяет свести мультипликативные эффекты к аддитивным

2. Взаимодействия параметров

Часто параметры совместно влияют на производительность. Например, влияние большой ширины W на временные задержки усугубляется при увеличении глубины дерева D .

Чтобы отразить совместное влияние этих признаков, в модель добавили произведения признаков:

- $D \cdot W = D \cdot W$
- $\log D \cdot \log W = \log D \cdot \log W$

Включение всех произведений признаков считаем нецелесообразным так, как высок риск переобучения, может ослабить нелинейность признаков и L2-регуляризация не сможет корректно выстроить веса

Анализ мультиколлинеарности и применение VIF

Поскольку признаки математически связаны, возникает проблема мультиколлинеарности (сильной линейной зависимости между предикторами). Для линейной регрессии это неприятно: веса модели становятся нестабильными

Для выявления сильно зависимых признаков используется фактор инфляции дисперсии (VIF). Для каждого признака x_j он рассчитывается по формуле:

$$VIF_j = \frac{1}{1 - R_j^2}$$

где R_j^2 - коэффициент детерминации линейной модели, в которой признак x_j выступает в роли целевой переменной, а все остальные признаки как в предикторы

Построение и обучение моделей регрессии

Масштабирование данных

Входные признаки имеют совершенно разные масштабы: объём памяти измеряется в байтах, коэффициент заполнения варьируется от 0 до 1, а логарифмы в единицах. Линейная регрессия чувствительна к масштабу, переменные с большими абсолютными значениями будут оказывать более сильное влияние на результат

Для устранения этого эффекта применяется стандартизация (Z-преобразование):

$$z = \frac{x - \mu}{\sigma}$$

где μ - среднее значение признака, а σ - стандартное отклонение

Использование цепочек регрессоров

В рамках исследования необходимо предсказать вектор из четырёх взаимосвязанных целевых метрик: объём памяти, среднюю задержку, 99-й перцентиль задержки и пропускную способность

Обучение независимых моделей для каждой метрики не учитывает физические связи внутри системы, например, то, что рост латентности снижает пропускную способность. Для решения этого использован подход цепочек регрессоров

Суть заключается в последовательном обучении регрессоров, где каждый следующий шаг принимает на вход не только матрицу признаков, но и предсказания всех предыдущих моделей

В качестве базового алгоритма внутри цепочки используется регрессия с L_2 -регуляризацией

$$L_{\text{Ridge}}(w) = \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i)^2 + \alpha \|w\|^2$$

Параметр α подбирается экспериментально и отвечает за подавление весов при избыточных признаках

Кросс-валидация и оценка качества

Исходный датасет разделяется в классическом соотношении: 80% данных отводится под обучение и валидацию, 20% под тестирование

Подбор гиперпараметра регуляризации α и порядок регрессоров проводятся с помощью кросс-валидации на 5 блоков. Обучающая выборка делится на 5 равных частей, модель последовательно обучается на 4 из них и проверяется на оставшейся части